ESERCITAZIONE 2 PCS MARCO - BUZZANTI OSSERVAZ

Nel mio caso, la prima variabile salvata in memoria è la y, definita float y = 1.1, salvata all'indirizzo 0x7ffc0bb581cc, successivamente salva la x in 0x7ffc0bb581d0 K tra i 2 indirizzi

Si può notare un salto di 4 byte (cc + 4 = d0 in esadecimale), perché il tipo float occupa 4 byte, come l'int, infatti, successivamente, l'array ai parte da un indirizzo base K1d4, che è cc + 4 + 4 = d0 + 4 = d4, poi, gli elementi i-esimi (0 ≤ i ≤ 2) sono salvati all'indirizzo base(ai) + i · sizeof(int) ⇒ &ai[0] = K1d4, ..., &ai[2] = K1d c

⇒ la base dell'array af è K1e0, e &af[0] = K1e0, ..., &af[7] = K1fc, come, siccome sizeof(float) = 4.

Invece per ad la base è K200, e gli indirizzi di memoria degli elem. si muovono di 8 in 8, siccome sizeof(double) = 8

indirizzo base K1cc K1c0 K200 8 byte

4 byte

...	y	x	ai[0]	ai[1]	ai[2]	af[0]	af[1]	af[2]	af[3]	af[4]	af[5]	af[6]	af[7]	ad[0]
	ad[1]		ad[2]			ad[3]								

indirizzo K220 (ogni cella è grande 4 byte)

Dalla stampa degli indirizzi è evidente che gli elementi degli array sono memorizzati in maniera contigua, in particolare x e y sono state allocate rispettivamente negli indirizzi K1c0 e K1cc, prima y e poi x! (con 4 byte di distanza, dovuti al sizeof(float))

L'operazione (&y) passa l'indirizzo di memoria di y, e il dato estratto è di tipo float* (è un puntatore). Ora, normalmente se ho un array A e voglio ottenere il valore 0 dell'i-esima componente si ha A[i] = 0, e come abbiamo visto sopra l'indirizzo di A[i] è base + i · sizeof(T), in questo caso base (&y)[1] = 0 è come dire: prendi la base &y (che è K1cc), somma i · sizeof(T) = 1 · sizeof(float) = 4 e a quell'indirizzo di memoria mettici il valore 0 (dereferenziazione). Infatti &x = &y + 4 !!

Quindi (&y)[1] va a sovrascrivere la x. La stampa 1 invece (penso) che probabilmente la x sia salvata prima di y (e magari sovrascrive ai[0]